

Lincoln University College
Faculty of Computer Science and Multimedia

**PocketDRS: A Single-View 3D Trajectory
Reconstruction and Decision Review System for Cricket**

MID DEFENSE REPORT

Submitted to
Department of Information Technology
Phoenix College of Management
Maitidevi, Kathmandu

In partial fulfillment of the requirements for the degree of
Bachelor in Information Technology (BIT)

Submitted by
Niraj Kafle
BIT 7th Semester
University ID: LC0003001674

March, 2026

Contents

1	Introduction	1
2	Problem Statement	1
3	Objectives	2
3.1	General Objective	2
3.2	Specific Objectives	2
4	Methodology	2
4.1	Requirement Identification	2
4.1.1	Study of Existing System	2
4.1.2	Literature Review	3
4.1.3	Requirement Analysis	3
4.2	Feasibility Study	4
4.2.1	Technical Feasibility	4
4.2.2	Operational Feasibility	4
4.2.3	Economic Feasibility	6
4.3	High Level Design of System	6
4.3.1	Context Level DFD	6
4.3.2	Level 1 DFD	6
4.3.3	Working Flow	6
4.3.4	System Components	6
4.3.5	Deployment View	10
4.3.6	Current Implementation Status	10
5	Gantt Chart	11
6	Expected Outcome	11
7	References	12

1 Introduction

Cricket officiating increasingly depends on technology for reviewing close decisions, especially Leg Before Wicket (LBW) cases. In elite matches, systems such as Hawk-Eye reconstruct the ball trajectory using multiple synchronized high-speed cameras and specialized processing infrastructure. Although these systems are highly effective, they are costly, complex to operate, and impractical for schools, clubs, local leagues, and training environments [1].

PocketDRS is proposed as a low-cost decision-support system that uses a single smartphone camera and a lightweight backend service to analyze a delivery. The user records or selects a short video clip, calibrates the visible pitch by marking known reference points, submits the delivery for processing, and receives a visual explanation together with an LBW assessment.

The project emphasizes practicality over broadcast-grade precision. The goal is not to replace certified professional DRS, but to demonstrate that a constrained single-view pipeline can provide meaningful review support in low-resource cricket settings. The current prototype focuses on reliable pitch-plane tracking, event detection, and explainable visualization, while acknowledging the accuracy limits of monocular analysis.

2 Problem Statement

Professional DRS systems depend on multiple synchronized cameras to recover depth and reconstruct the trajectory of the ball in three dimensions. This produces strong results, but the required equipment, installation effort, and operating cost make such systems inaccessible outside major venues.

When the same decision-review problem is shifted to a single consumer camera, several technical and practical difficulties arise:

1. A monocular view does not provide direct depth information.
2. A cricket ball is small, fast, motion-blurred, and can easily be confused with background objects.
3. Small calibration errors can produce large errors when detections are mapped to pitch coordinates.
4. Lighting variation, camera shake, player occlusion, and inconsistent recording conditions reduce reliability in real grounds.

Therefore, the central problem addressed by this project is whether a single-phone-camera workflow, supported by pitch calibration and lightweight computer-vision techniques, can produce a sufficiently consistent and understandable LBW review aid for ordinary cricket grounds.

3 Objectives

3.1 General Objective

To design and implement a low-cost cricket decision-support system that uses a single camera view to analyze a delivery, estimate the ball path in calibrated pitch coordinates, and assist LBW review.

3.2 Specific Objectives

1. Study existing decision-review systems and relevant literature on monocular sports tracking.
2. Develop a user-guided calibration process based on pitch corners and stump reference points.
3. Detect and track the ball from delivery video using combined motion and colour cues.
4. Map tracked pixel positions to pitch-plane coordinates using homography.
5. Detect important events such as bounce and impact from the tracked trajectory.
6. Estimate an LBW outcome using rule-based checks on pitching line, impact line, and projected stump contact.
7. Present the result through a simple interactive visualization that helps users interpret the decision.

4 Methodology

4.1 Requirement Identification

4.1.1 Study of Existing System

Existing systems such as Hawk-Eye use multiple calibrated cameras, controlled camera placement, high frame rates, and proprietary reconstruction pipelines. These systems are

designed for professional broadcasting environments. In contrast, PocketDRS adopts a constrained workflow built for affordability: one mobile camera, one short trimmed delivery clip, one user-guided calibration step, and a lightweight server-side analysis pipeline. The trade-off is lower accuracy than professional systems, but much lower deployment cost and complexity.

4.1.2 Literature Review

Related research shows that single-view sports analysis becomes more feasible when the scene contains known geometry and the target motion follows domain constraints. In cricket, the pitch provides stable reference landmarks, while the ball generally follows a predictable sequence from release to bounce, then impact, and finally the projected wicket line. Homography-based mapping from image coordinates to a known plane is a practical way to convert 2D observations into real-world pitch coordinates [3]. Background subtraction and colour filtering remain effective low-cost detection strategies in controlled or semi-controlled environments, especially when the search space is reduced through domain knowledge [4]. Recent work on monocular ball trajectory estimation further suggests that strong constraints can compensate for missing depth information, although monocular systems remain more fragile than multi-camera solutions [2]. These findings support the feasibility of a student-scale prototype that prioritizes explainability and affordability.

4.1.3 Requirement Analysis

Functional requirements

1. User signs in.
2. User records or selects a video.
3. User creates or reuses pitch calibration.
4. User trims the clip and submits it.
5. Server tracks the ball through the selected segment.
6. Server maps the track to pitch coordinates.
7. Server detects bounce and impact points.
8. Server evaluates the LBW outcome.
9. App shows the decision, warnings, and an interactive visual result.

Non-functional requirements

1. The system should stay easy to use for a first-time user.
2. Processing should finish in a short time for one delivery.
3. The app should run on both Android and iOS.
4. The server should work with common phone video quality and practical frame rates.
5. The final result should be understandable to non-technical users.
6. The system should provide diagnostic warnings when calibration or tracking quality is weak.
7. The architecture should remain modular so that stronger tracking or reconstruction methods can be added later.

4.2 Feasibility Study

4.2.1 Technical Feasibility

The proposed system is technically feasible because it is built from mature and widely supported technologies. Flutter provides cross-platform mobile development, FastAPI provides lightweight and testable backend services, OpenCV supports frame processing and tracking, and Firebase handles authentication and cloud persistence. The present implementation already demonstrates upload, calibration, server-side analysis, and result visualization on consumer devices. Although monocular video imposes accuracy limits, the required software stack is realistic for a student project and deployable on ordinary hardware.

4.2.2 Operational Feasibility

The operational workflow is simple enough for field use during testing and demonstrations. A user places a phone near square leg or a side-on view, records a delivery, calibrates the pitch by tapping known points, trims the clip, and waits for the generated result. This procedure is repeatable, requires minimal training, and fits the needs of clubs, student projects, and coaching sessions.

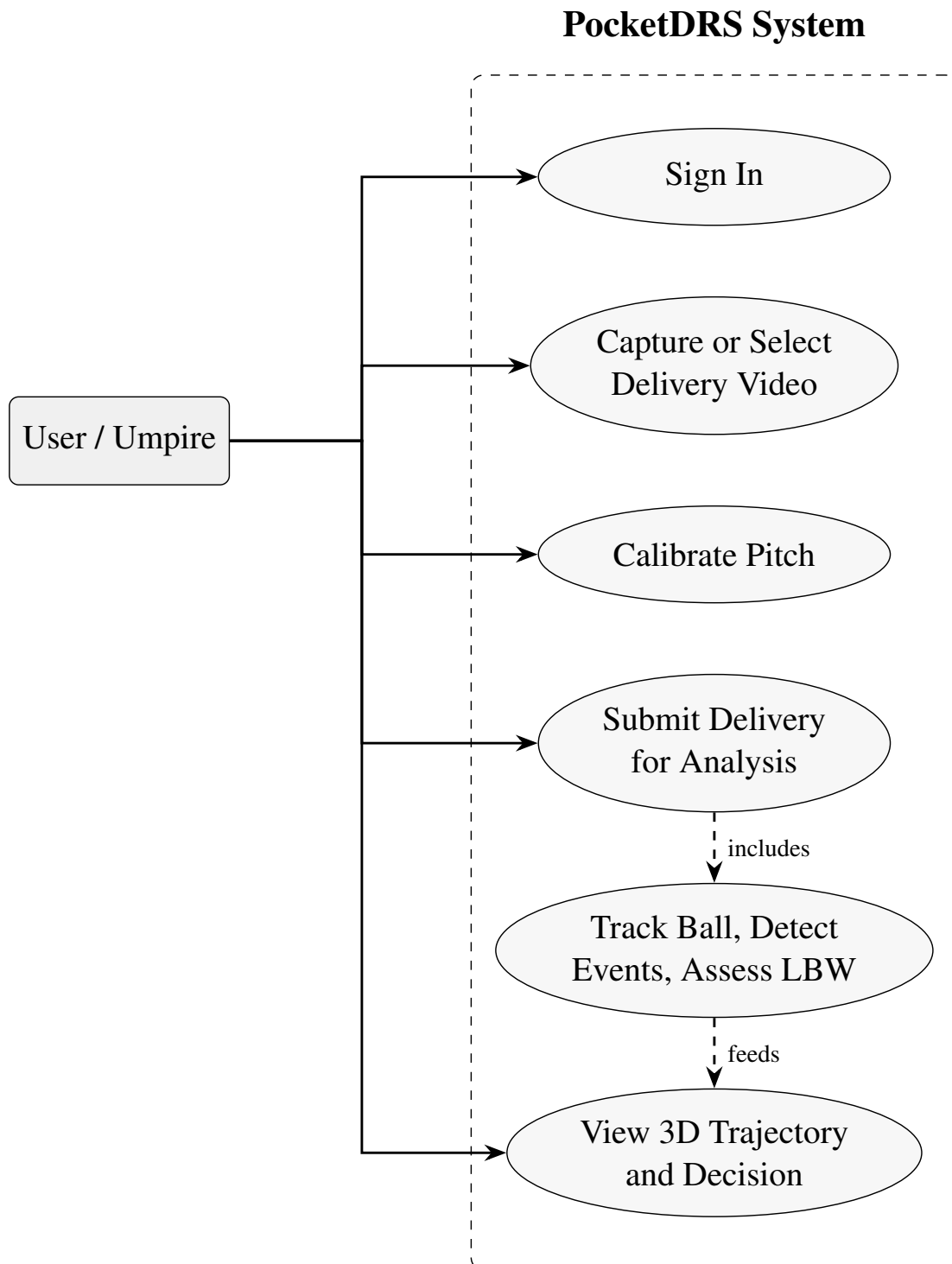


Figure 1: Main use cases of PocketDRS.

4.2.3 Economic Feasibility

The project is economically feasible because it does not require specialized camera arrays, licensed broadcast systems, or custom hardware. It relies primarily on a smartphone, a tripod, and open-source libraries. Compared with professional DRS infrastructure, the expected deployment cost is extremely low, making the proposed system suitable for academic demonstration and low-budget cricket environments.

Table 1: Main tools used in PocketDRS.

Part	Tool
Mobile app	Flutter and Dart
Server API	FastAPI and Python
Vision pipeline	OpenCV, NumPy, SciPy
Authentication	Firestore Authentication
Data storage	Cloud Firestore
3D result view	Three.js inside WebView
Testing	pytest and Flutter analyzer

4.3 High Level Design of System

The proposed system is designed as a mobile-to-server workflow. The mobile application handles video capture or selection, pitch calibration, job submission, and result presentation. The backend decodes the selected segment, tracks the ball, computes a homography from user-provided pitch points, maps detections to pitch coordinates, detects key events, evaluates the LBW decision, and returns structured output to the client. The following diagrams summarize the architecture, data flow, software components, deployment structure, and working mechanism of the system.

4.3.1 Context Level DFD

4.3.2 Level 1 DFD

4.3.3 Working Flow

The working flow is kept short so that users can perform the analysis with minimal overhead:

4.3.4 System Components

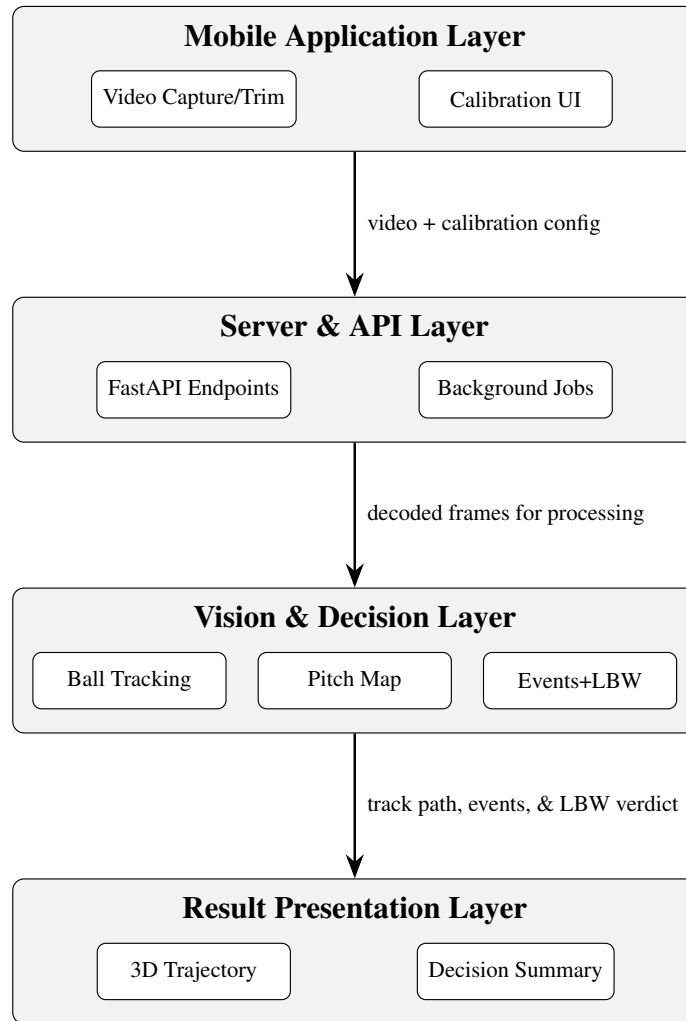


Figure 2: Overall architecture of PocketDRS.

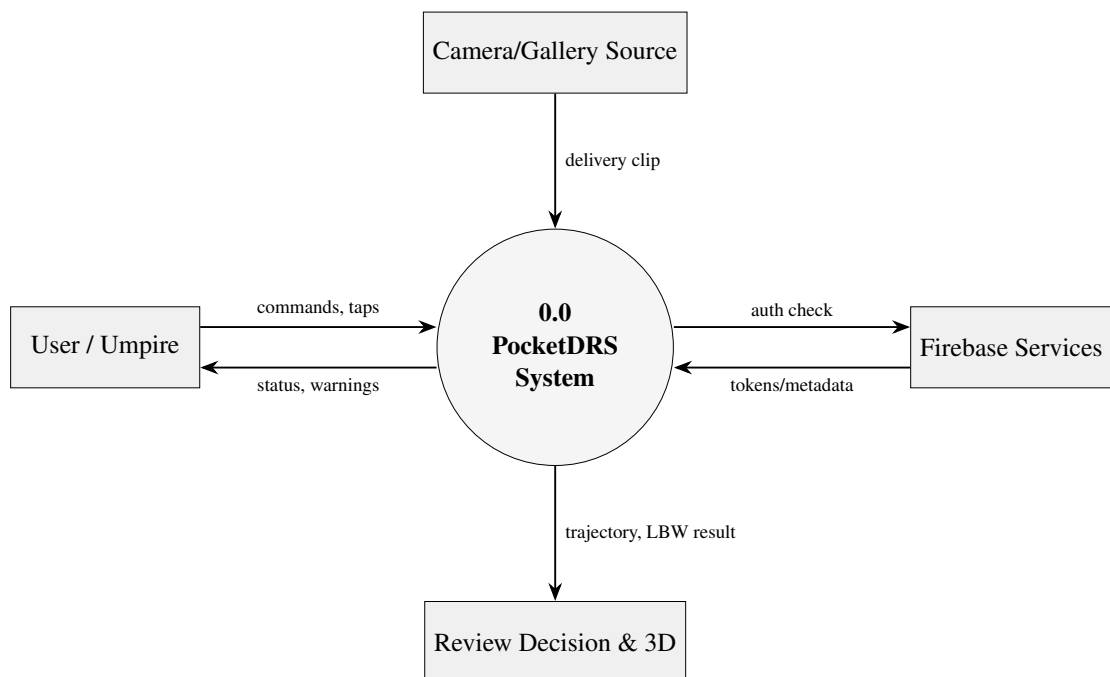


Figure 3: Context-level DFD of PocketDRS.

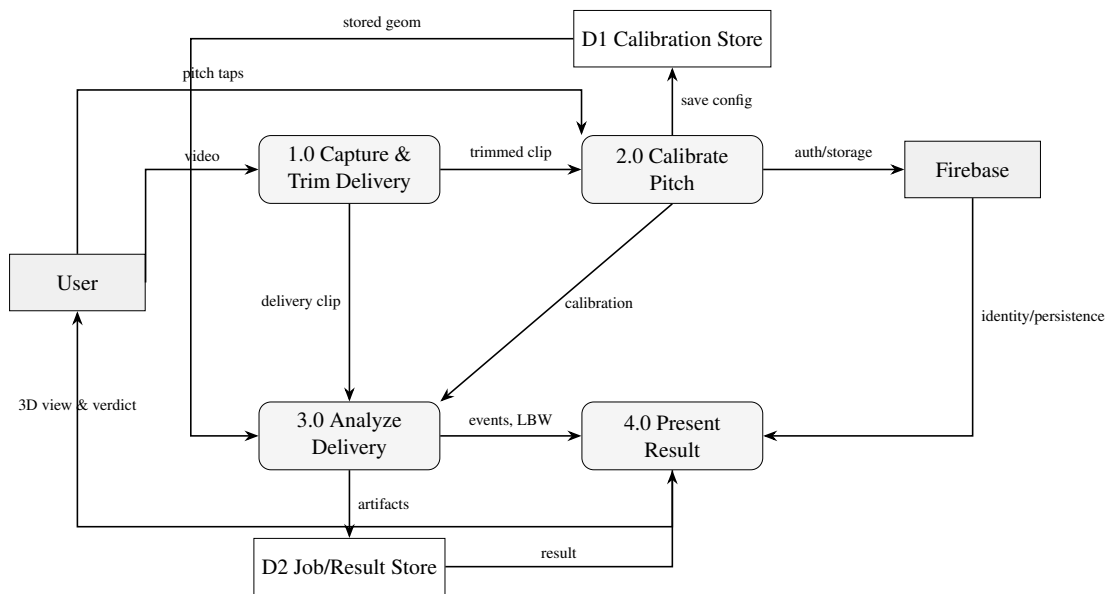


Figure 4: Level-1 DFD of the PocketDRS processing workflow.

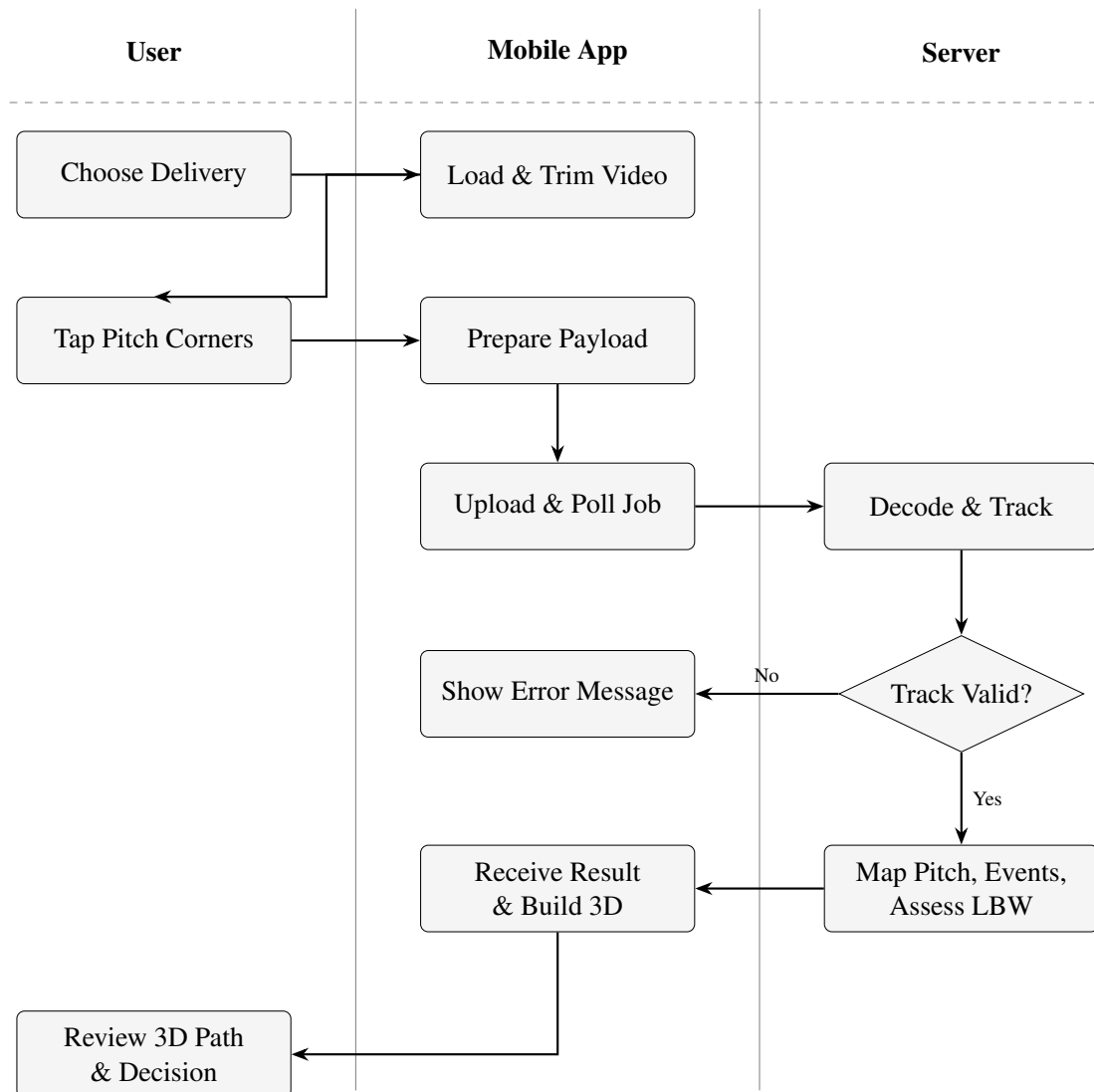


Figure 5: Main processing flow used by the system.

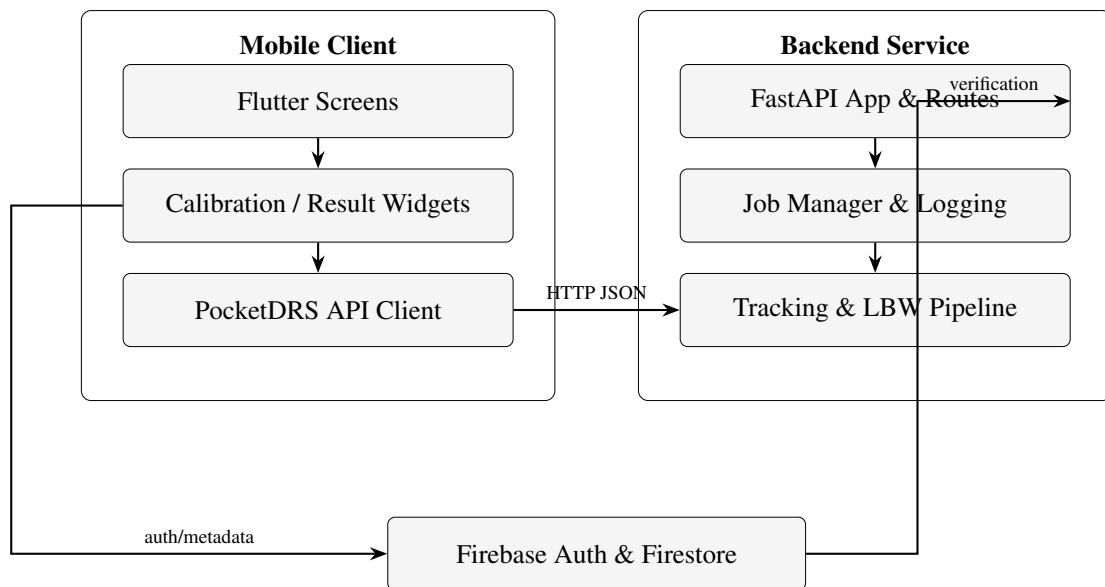


Figure 6: Main software components.

4.3.5 Deployment View

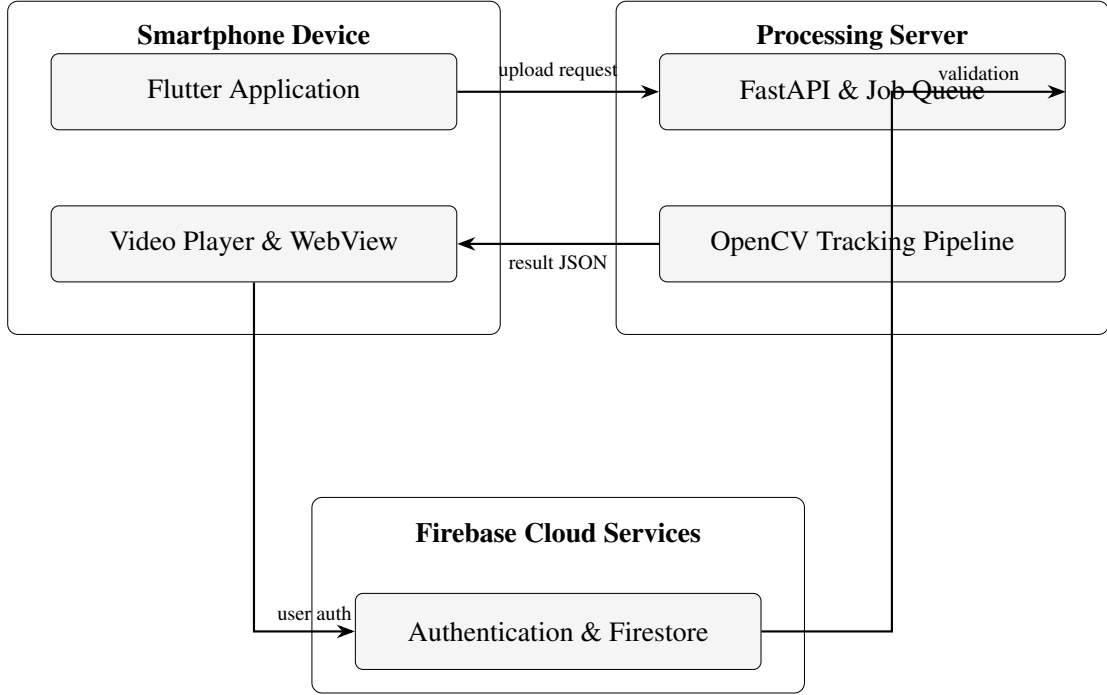


Figure 7: Deployment view of the working system.

4.3.6 Current Implementation Status

The project has progressed beyond initial planning and currently exists as a working prototype. The following summary reflects the present implementation state rather than the long-term vision of the project.

Mobile Application. The Flutter app supports Firebase sign-in, video selection or recording, an interactive pitch calibration step, video trimming, job submission, progress polling, and result display. The calibration workflow collects four pitch corners and stump reference points and validates the geometry before submission.

Server and API. The FastAPI backend accepts authenticated uploads and processes them as background jobs. Each job performs frame decoding, ball tracking, calibration mapping, event estimation, and LBW assessment. The API returns structured JSON including track points, calibration quality, pitch-plane points, bounce and impact indices, decision details, and diagnostic warnings.

Ball Tracking Pipeline. Tracking uses a combined motion-and-colour strategy. A motion detector based on background subtraction is combined with HSV colour filtering so that detections supported by both cues receive higher confidence. A pitch ROI mask derived from the calibration corners restricts the search region and reduces false positives from players, spectators, and background motion.

Pitch Calibration and Homography. The user marks four pitch corners and stump reference points on a reference frame. The backend computes a homography that maps image coordinates to pitch-plane coordinates in meters. When stump points are available, they are used to refine the mapping. A reprojection-based quality score is generated so that low-quality calibration can be flagged.

Event Detection and LBW. Bounce and impact are estimated primarily from the pitch-plane trajectory after calibration. The LBW module then applies rule-based checks for pitching line, impact line, and projected stump contact. The current implementation is a calibrated 2D pitch-plane approximation intended for decision support rather than certified officiating.

Visualization Layer. The result view uses Three.js inside a WebView to display the pitch, stumps, key trajectory points, and decision summary. At the current stage, the visualization is primarily explanatory: the displayed height component is synthesized for presentation, while the core analytical decision is computed from calibrated pitch-plane data.

Scope and Limitations. The current prototype is appropriate for research demonstration, supervised testing, and low-cost review support. It is not ICC-certified, is sensitive to camera placement and lighting, and may fail when the ball is heavily occluded or the calibration input is poor. Full monocular 3D reconstruction remains a future enhancement rather than a completed feature in the current build.

5 Gantt Chart

Table 2: Project schedule.

Task	Dec	Jan	Feb	Mar	Apr
Problem identification and literature review	X	X			
Requirement analysis and system design		X	X		
Mobile app and backend implementation		X	X	X	
Tracking, calibration, and LBW logic			X	X	
Testing, debugging, and refinement				X	X
Documentation and mid/final report updates				X	X

6 Expected Outcome

The expected outcome of PocketDRS is a working prototype of a low-cost cricket review-support tool that can:

1. Accept a single delivery clip from a phone.

2. Calibrate the visible pitch using user-selected reference points.
3. Track the ball through the important part of the delivery.
4. Map the tracked detections onto pitch-plane coordinates.
5. Estimate bounce and impact points with diagnostic feedback.
6. Produce an interpretable LBW decision with a visual explanation.
7. Demonstrate that a practical DRS-like workflow is possible on a small budget.

In addition, the project is expected to establish a foundation for future work such as stronger ball detectors, more robust calibration, improved prediction models, and true monocular 3D reconstruction. Even in its current form, the system is valuable as a research prototype, a teaching aid, and a demonstration of affordable sports-analytics workflow design.

7 References

References

- [1] Hawk-Eye Innovations, “Hawk-Eye: The Technology Behind Sports Officiating,” 2024. [Online]. Available: <https://www.hawkeyeinnovations.com/>
- [2] P. Ponglertnapakorn, T. Shirai, and T. Yamasaki, “Where Is the Ball: 3D Ball Trajectory Estimation From 2D Monocular Tracking,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [3] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [4] OpenCV Documentation, “Camera Calibration and 3D Reconstruction,” 2024. [Online]. Available: <https://docs.opencv.org/>
- [5] International Cricket Council, “Playing Conditions: Law 36 Leg Before Wicket,” 2024.
- [6] Google, “Flutter: Build apps for any screen,” 2024. [Online]. Available: <https://flutter.dev/>
- [7] S. Ramírez, “FastAPI: Modern, fast web framework for building APIs with Python,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [8] Three.js Contributors, “Three.js: JavaScript 3D Library,” 2024. [Online]. Available: <https://threejs.org/>